

Conociendo los servlets

M5: Desarrollo de aplicaciones web dinámicas en Java

|AE3: Estructurar el comportamiento de una aplicación web dinámica utilizando Servlets que procesan peticiones GET y POST para dar solución a un problema

Introducción

En esta lección, aprenderemos sobre los **Servlets y los Contenedores Web**, tecnologías esenciales **para construir aplicaciones web dinámicas y eficientes**.

Vamos a familiarizarnos con Sesiones y Cookies, y cómo utilizarlos para mantener información del usuario entre diferentes solicitudes. También veremos cómo compartir datos entre Servlets para crear aplicaciones cohesivas y escalables.

Además, estudiaremos la concurrencia con Servlets, asegurando la seguridad y estabilidad de nuestras aplicaciones en entornos multiusuario. Aprenderemos a controlar los parámetros de las solicitudes GET y POST, permitiéndonos recibir y procesar datos del cliente.

También aprenderemos cómo enviar parámetros desde un Servlet hacia una vista JSP, lo que nos permitirá separar la lógica de presentación y crear páginas web dinámicas.

Aprendizaje esperado

Al finalizar la lección podrás:

- Estructurar el comportamiento de una aplicación web dinámica utilizando Servlets que procesan peticiones GET y POST para dar solución a un problema.

Servlets

Un servlet es un **programa Java que se ejecuta en un servidor Web y construye o sirve páginas web**. De esta forma se pueden construir páginas dinámicas, basadas en diferentes fuentes variables: datos proporcionados por el usuario, fuentes de información variable (páginas de noticias, por ejemplo), o programas que extraigan información de bases de datos.

Los servlets son sencillos de utilizar, más eficientes (se arranca un hilo por cada petición y no un proceso entero), más potente y portable. Con los servlets podremos, entre otras cosas, procesar, sincronizar y coordinar múltiples peticiones de clientes, reenviar peticiones a otros servlets u otros servidores, etc.

¿Cuál es la función de un Servlet?

1. Tomar la solicitud del cliente y devolver una respuesta.
2. La solicitud adjunta datos, y el código del Servlet sabe como recuperarlos y como usarlos.
3. La respuesta adjunta la información que el navegador necesita para mostrar la página, y el Servlet es quien envía esta información.
4. El Servlet puede decidir enviar la solicitud a otra página, Servlet o JSP.

Contenedor web de aplicaciones

Un contenedor web es la implementación que hace cumplimiento del contrato de componentes web de la arquitectura J2EE. Este contrato es un entorno de ejecución para componentes web que incluye seguridad, concurrencia, gestión del ciclo de vida, procesamiento de transacciones, despliegue y otros servicios. Un contenedor web se suministra incluido en un servidor web o J2EE. El ejemplo más cercano es Apache Tomcat.

Arquitectura del paquete servlet

Dentro del paquete `javax.servlet` tenemos toda la infraestructura para poder trabajar con servlets. El elemento central es la **interfaz Servlet**, que define los métodos para cualquier servlet. La clase `GenericServlet` es una clase abstracta que implementa dicha interfaz para un servlet genérico, independiente del protocolo. Para definir un servlet que se utilice vía web, se tiene la clase `HttpServlet` dentro del subpaquete `javax.servlet.http`. Esta clase hereda de

GenericServlet, y también es una clase abstracta, de la que heredaremos para construir los servlets para nuestras aplicaciones web.

Cuando un servlet acepta una petición de un cliente, se reciben dos objetos:

1. Un objeto de tipo **ServletRequest** que contiene los datos de la petición del usuario (toda la información entrante). Con esto se accede a los parámetros pasados por el cliente, el protocolo empleado, etc. Se puede obtener también un objeto **ServletInputStream** para obtener datos del cliente que realiza la petición. La subclase **HttpServletRequest** procesa peticiones de tipo HTTP.
2. Un objeto de tipo **ServletResponse** que contiene (o contendrá) la respuesta del servlet ante la petición (toda la información saliente). Se puede obtener un objeto **ServletOutputStream**, y un **Writer**, para poder escribir la respuesta. La clase **HttpServletResponse** se emplea para respuestas a peticiones HTTP.

Ciclo de vida de un servlet

Todos los servlets tienen el mismo ciclo de vida:

- Un servidor carga e inicializa el servlet.
- El servlet procesa cero o más peticiones de clientes (por cada petición se lanza un hilo).
- El servidor destruye el servlet (en un momento dado o cuando se apaga).

1. Inicialización

Los servlet tienen un inicializador por defecto en el método **init()**.

```
public void init() throws ServletException{  
    ...  
}  
  
public void init(ServletConfig conf) throws ServletException{  
    super.init(conf);  
    ...  
}
```

El primer método se utiliza si el servlet no necesita parámetros de configuración externos. El segundo se emplea para tomar dichos parámetros del objeto **ServletConfig** que se le pasa. La llamada a **super.init(...)** al principio del método es MUY importante, porque el servlet utiliza esta configuración en otras zonas.

Si queremos definir nuestra propia inicialización, deberemos **sobreescibir** alguno de estos métodos. Si ocurre algún error al inicializar y el servlet no es capaz de atender peticiones, debemos lanzar una excepción de tipo **UnavailableException**.

Podemos utilizar la inicialización para establecer una conexión con una base de datos (si trabajamos con base de datos), abrir ficheros, o cualquier tarea que se necesite hacer una sola vez antes de que el servlet comience a funcionar.

2. Procesamiento de peticiones

Una vez inicializado, cada petición de usuario lanza un hilo que llama al método **service()** del servlet.

```
public void service(HttpServletRequest request, HttpServletResponse response)  
throws ServletException, IOException
```

Este método obtiene el tipo de petición que se ha realizado (GET, POST, PUT, DELETE). Dependiendo del tipo de petición que se tenga, se llama luego a uno de los métodos:

doGet(): Para peticiones de tipo **GET** (aquellas realizadas al escribir una dirección en un navegador, pinchar un enlace o rellenar un formulario que no tenga METHOD=POST)

```
public void doGet(HttpServletRequest request,  
                  HttpServletResponse response)
```

```
throws ServletException, IOException
```

doPost(): Para peticiones POST (aquellas realizadas al rellenar un formulario que tenga METHOD=POST)

```
public void doPost(HttpServletRequest request,  
                  HttpServletResponse response)
```

```
throws ServletException, IOException
```

doXXX(): normalmente sólo se emplean los dos métodos anteriores, pero se tienen otros métodos para peticiones de tipo DELETE (**doDelete()**), PUT (**doPut()**), OPTIONS (**doOptions()**) y TRACE (**doTrace()**).

3. Destrucción

El método `destroy()` de los servlets se emplea para eliminar un servlet y sus recursos asociados.

`public void destroy() throws ServletException`

Aquí debe deshacerse cualquier elemento que se construyó en la inicialización (cerrar conexiones con bases de datos, cerrar ficheros, etc). El servidor llama a `destroy()` cuando todas las llamadas de servicios del servlet han concluido, o cuando haya pasado un determinado número de segundos (lo que ocurra primero).

Estructura básica de un servlet

La plantilla común para implementar un servlet es:

```
import javax.servlet.*;
import javax.servlet.http.*;

public class ClaseServlet extends HttpServlet
{
    public void doGet(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // ... código para una petición GET
    }

    public void doPost(HttpServletRequest request,
                      HttpServletResponse response)
        throws ServletException, IOException
    {
        // ... código para una petición POST
    }
}
```

Mapeado de servlets y páginas JSP en el fichero descriptor

La opción más utilizada para llamar al servlet es incluir en el fichero descriptor de la aplicación (web.xml en Tomcat) la información necesaria para que lo encuentre. Dicha información consiste en introducir una marca `<servlet>` para cada servlet que se quiera llamar de esta forma:

`<servlet>`

`<servlet-name>nombre</servlet-name>`

`<servlet-class>ClaseServlet</servlet-class>`

</servlet>

Donde **<servlet-name>** es un nombre identificativo y arbitrario del servlet, y **<servlet-class>** es la ruta a la clase del servlet (incluyendo paquetes y subpaquetes, separados por '.'). Con esto, al servlet `ClaseServlet` lo podemos llamar de dos formas:

`http://localhost:8080/<dir>/servlet/ClaseServlet`

`http://localhost:8080/<dir>/servlet/nombre`

Siendo **<dir>** el directorio de la aplicación Web. De forma similar se podría mapear una página JSP, sustituyendo la etiqueta **<servlet-class>** por la etiqueta **<jsp-file>**:

<servlet>

<servlet-name>nombre2</servlet-name>

<jsp-file>/miPagina.jsp</servlet-class>

</servlet>

Asignar URLs a servlets o páginas JSP

El uso de la ruta `.../servlet/...` para llamar a los servlets puede ser útil durante la depuración, pero luego podemos querer invocar al servlet utilizando una URL alternativa. Esto se consigue mediante las etiquetas **<servlet-mapping>**:

<servlet-mapping>

<servlet-name>nombre</servlet-name>

<url-pattern>/ejemploServlet</url-pattern>

</servlet-mapping>

En la subetiqueta **<servlet-name>** se pone el nombre del servlet al que se quiere asignar la URL (será uno de los nombres dados en alguna etiqueta **<servlet>** previa), y en **<url-pattern>** colocamos la URL que le asignamos al servlet, relativa a la raíz de la aplicación web, comenzando con '/'.

Notar que **primero** se colocan todas las etiquetas **<servlet>**, y luego las **<servlet-mapping>** que se requieran.

Así, con lo anterior, podremos llamar al servlet identificado con nombre de otra forma más: `http://localhost:8080/<dir>/ejemploServlet`

También podemos asignar en `<url-pattern>` expresiones como:

```
<servlet-mapping>  
    <servlet-name>nombre</servlet-name>  
    <url-pattern>/ejemploservlet/*</url-pattern>  
</servlet-mapping>
```

o como:

```
<servlet-mapping>  
    <servlet-name>nombre</servlet-name>  
    <url-pattern>/ejemploServlet/*.jsp</url-pattern>  
</servlet-mapping>
```

Con el primero, cualquier URL del directorio de nuestra aplicación Web que comience con `/ejemploservlet/` se redirigirá y llamará al servlet identificado con nombre. Por ejemplo, las direcciones:

Con el segundo, cualquier llamada a cualquier página JSP del directorio `/ejemploservlet/` de nuestra aplicación se redirigirá al servlet nombre.

Podemos hacer que distintas URLs llamen a un mismo servlet, sin más que añadir varios grupos `<servlet-mapping>`, uno por cada patrón de URL diferente, y todos con el mismo `<servlet-name>`.

Nos puede interesar que a los servlets no se les llame a través del alias `/servlet/*`. Para ello, podemos aplicar este procedimiento para mapear rutas con el alias `/servlet/..` para que se redirija a un servlet que muestre un mensaje de error. Este mismo procedimiento se aplica, sin cambio alguno, si en lugar de un servlet queremos tratar una página JSP.

Manejo de sesiones y cookies en Servlets Java

Gracias a la variedad de APIs que Java expone en sus paquetes nativos, nos ofrece gran facilidad y claridad de lenguaje al gestionar las cookies y las sesiones de nuestras aplicaciones web.

Manejando las Cookies

Crear una cookie es bastante fácil cuando utilizamos la clase `Cookie` que encontramos en el paquete `javax.servlet.http`, simplemente hacemos una instancia de esta clase y la mandamos en la respuesta del servidor con el método `addCookie()`.

```
private static final void galleta(HttpServletRequest request, HttpServletResponse response) {  
  
    Cookie galletaColor = new Cookie("color", "rojo");  
  
    response.addCookie(galletaColor);  
  
}
```

En este ejemplo podemos ver una cookie simple con todas sus opciones por defecto. Además, una cookie puede tener los siguientes parámetros:

Dominio: Establece el nombre de dominio (o dominios) para el cual una cookie es válida. Se establece a través del método `setDomain(String dominio)` de la clase `Cookie`.

Path: Establece la uri especifica en la que la cookie será válida (e.g: /admin). Es útil si no queremos exponer la cookie en toda la aplicación. Se establece con el método `setPath(String path)` de la clase `Cookie`.

Expiración: Indica cuánto debe durar una cookie en el navegador. Este valor se establece en segundos a través del método `setMaxAge(int segundos)` de la clase `Cookie`.

HostOnly: Esta propiedad es muy importante en términos de seguridad, ya que establece si una cookie podrá ser leída por el cliente o solo por el servidor. Piensen que si alguien logra un ataque XSS bien podrían robar las cookies de sus usuarios sin mayores pormenores. Por eso `HostOnly` debería ser `true` siempre que el dominio desde el cual se está intentando leer no sea igual al dominio de origen establecido.

Secure: Otra parámetro muy importante que indica que la cookie en cuestión puede ser leída únicamente a través de un protocolo seguro HTTPS o SSL. Y se establece con el método `setSecure(boolean isSecure)` de la clase `Cookie`.

HttpOnly: Es similar a `HostOnly` excepto que esta si se puede manipular. Funciona como una medida de seguridad extra a la especificación establecida. Java al ser un lenguaje de clase mundial ofrece el método `setHttpOnly(boolean isHttpOnly)` para estos efectos.

Manejando Las Sesiones

Una buena alternativa para no manejar información sensible de nuestros usuarios del lado del cliente son las sesiones, que no es otra cosa que una estructura de datos que es accedida exclusivamente del lado del servidor. Si sus sistemas son muy concurridos o están detrás de un balanceador de carga, manejar sesiones podría representar un reto interesante de resolver, sin embargo, la mayoría de las veces funciona muy bien con su configuración por defecto.

Veamos un ejemplo práctico de cómo manipular la sesión en nuestro servlet.

```
private static final void home(HttpServletRequest request, HttpServletResponse response) {
```

```
    HttpSession sesion = request.getSession();  
  
    session.setAttribute("usuario", "estudiante");
```

```
}
```

Hemos creado una sesión y le hemos insertado un registro que describe un nombre de usuario. Luego podemos acceder a esta sesión en un **request** completamente diferente.

```
private static final void panel(HttpServletRequest request, HttpServletResponse response) {
```

```
    HttpSession sesion = request.getSession();  
  
    Object usuario = (String) session.getAttribute("usuario");
```

```
}
```

Aquí obtenemos el nombre de usuario. Vemos que como el **getAttribute** devuelve un objeto lo casteamos al tipo correcto para su posterior utilización.

Finalmente si quisiéramos borrar (invalidar) la sesión, podemos hacerlo de la siguiente manera:

```
private static final void salir(HttpServletRequest request, HttpServletResponse response) {
```

```
    HttpSession sesion = request.getSession();  
  
    session.invalidate();
```

```
}
```

El método `invalidate()` destruye la sesión y sus contenidos.

Compartiendo información entre Servlets

Algunas razones por las que es útil compartir información entre servlets:

- **Para simplificar el código:** se puede evitar tener que pasar la información repetidamente entre los servlets. Esto puede simplificar el código y hacerlo más fácil de mantener.
- **Para mejorar el rendimiento:** se puede evitar tener que volver a cargar la información de la base de datos o de otros recursos. Esto puede mejorar el rendimiento de la aplicación.
- **Para aumentar la seguridad:** se puede restringir el acceso a la información a los usuarios autorizados. Esto puede ayudar a proteger su información contra el acceso no autorizado.

En general, compartir información entre servlets puede ser una forma eficaz de mejorar la funcionalidad, el rendimiento y la seguridad de su aplicación.

Algunas formas de compartir información entre servlets J2EE:

Atributos del contexto: son una forma de compartir información entre todos los servlets en una aplicación web. Los atributos del contexto de aplicación son compartidos por todos los servlets y JSP que se ejecutan en la misma aplicación en el servidor.

Para agregar un atributo al contexto, se utiliza el método `setAttribute()` en el objeto `ServletContext`. Para obtener un atributo del contexto, se utiliza el método `getAttribute()`.

```
// En el primer servlet
String dato = "Hola desde el primer servlet";
ServletContext context = getServletContext();
context.setAttribute("datoCompartido", dato);
```

```
// En el segundo servlet (segundoServlet)
ServletContext context = getServletContext();
String dato = (String) context.getAttribute("datoCompartido");
```

Sesiones: Las sesiones son una forma de compartir información entre solicitudes de un mismo usuario. Los atributos de sesión son específicos para cada usuario y permanecen disponibles durante toda la sesión, incluso si el usuario navega entre diferentes páginas o servlets.

Para crear una sesión, se utiliza el método `getSession()` en el objeto `HttpServletRequest`. Para agregar información a una sesión, se utiliza el método `setAttribute()` en el objeto `HttpSession`. Para obtener información de una sesión, se utiliza el método `getAttribute()`.

```
// En el primer servlet
String dato = "Hola desde el primer servlet";
HttpSession session = request.getSession();
session.setAttribute("datoCompartido", dato);
```

```
// En el segundo servlet (segundoServlet)
HttpSession session = request.getSession();
String dato = (String) session.getAttribute("datoCompartido");
```

Atributos de solicitud: Puedes compartir información entre servlets a través de atributos de solicitud. Cuando un servlet envía una solicitud a otro servlet o JSP, puede establecer atributos en la solicitud que serán accesibles por el servlet de destino.

```
// En el primer servlet
String dato = "Hola desde el primer servlet";
request.setAttribute("datoCompartido", dato);
RequestDispatcher dispatcher = request.getRequestDispatcher("/segundoServlet");
dispatcher.forward(request, response);
```

```
// En el segundo servlet (segundoServlet)
String dato = (String) request.getAttribute("datoCompartido");
```

Concurrencia entre Servlets

La concurrencia entre servlets es un tema importante a tener en cuenta cuando trabajas con aplicaciones web, especialmente cuando varios hilos pueden interactuar con los mismos servlets al mismo tiempo. La **concurrencia** se refiere a la **ejecución simultánea de múltiples hilos o procesos que intentan acceder y modificar recursos compartidos**.

En el contexto de los servlets, la concurrencia se puede presentar en varias situaciones:

1. **Solicitudes concurrentes:** Cuando varios clientes (navegadores u otras aplicaciones) envían solicitudes HTTP al mismo servlet al mismo tiempo. Cada solicitud se maneja en un hilo separado en el servidor, lo que significa que múltiples hilos pueden ejecutar el mismo servlet simultáneamente.
2. **Recursos compartidos:** Si varios servlets comparten recursos globales, como variables estáticas o atributos de contexto de aplicación, podría haber problemas de concurrencia cuando varios hilos intentan acceder y modificar esos recursos compartidos al mismo tiempo.
3. **Sesiones:** Cuando varios usuarios interactúan con la misma aplicación web y utilizan sesiones para mantener información específica del usuario, es posible que varios hilos intenten acceder y modificar la misma sesión al mismo tiempo.

Los problemas de concurrencia pueden llevar a condiciones de carrera, bloqueos, inconsistencias de datos y otros errores inesperados en la aplicación. Para manejar la concurrencia de manera adecuada, es esencial implementar mecanismos para garantizar la seguridad y coherencia de los recursos compartidos.

Algunas estrategias para manejar la concurrencia en servlets incluyen:

- **Sincronización:** Utiliza bloqueos (`synchronized`) para garantizar que solo un hilo pueda acceder a recursos compartidos en un momento dado. Esto asegura que un recurso no sea modificado por varios hilos al mismo tiempo, evitando condiciones de carrera y bloqueos.
- **Evitar recursos compartidos:** Diseña tu aplicación para evitar el uso excesivo de recursos compartidos. En lugar de compartir variables globales, considera el uso de atributos de solicitud o de sesión para mantener la información específica del usuario.
- **Uso de estructuras de datos concurrentes:** Si es necesario utilizar recursos compartidos, utiliza estructuras de datos concurrentes, como

`ConcurrentHashMap`, que están diseñadas para manejar la concurrencia de manera segura.

- **Uso de sesiones seguras:** Asegúrate de que las sesiones sean seguras para ser accedidas y modificadas por múltiples hilos al mismo tiempo. Utiliza las sesiones correctamente y evita la posibilidad de pérdida de datos o inconsistencias.
- **Pruebas y análisis:** Realiza pruebas exhaustivas para identificar posibles problemas de concurrencia. Utiliza herramientas de análisis de concurrencia y asegúrate de que tu aplicación funcione de manera correcta y predecible en entornos concurrentes.

Controlando parámetros de un GET request

Cuando los clientes realizan una solicitud GET, los parámetros se envían en la URL y se pueden acceder desde el servlet para procesar la solicitud. Para controlar los parámetros en un GET request, es necesario seguir estos pasos:

1. Obtén el valor de los parámetros en el servlet en el método `doGet()` utilizando el objeto `HttpServletRequest`. Puedes acceder a los parámetros mediante los métodos `getParameter()` o `getParameterValues()`.
2. Procesa y valida los parámetros según sea necesario para tu aplicación.

Veamos un ejemplo de cómo controlar los parámetros de una solicitud GET en un servlet:

Supongamos que tienes un servlet llamado `MyServlet` que manejará una solicitud GET con dos parámetros, "nombre" y "edad". El servlet mostrará estos parámetros en la respuesta al cliente.

```
@WebServlet("/mainServlet")
public class MainServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        // Obtenemos el valor del parámetro "nombre" de la solicitud GET
        String nombre = request.getParameter("nombre");

        // Obtenemos el valor del parámetro "edad" de la solicitud GET
        String edad = request.getParameter("edad");

        // Procesamos y validamos los parámetros según sea necesario
        // Por ejemplo, podríamos verificar si los parámetros son nulos o vacíos

        // Creamos la respuesta para el cliente
        response.setContentType("text/html;charset=UTF-8");
        response.getWriter().println("<html><body>");
        response.getWriter().println("<h1>Información recibida:</h1>");
        response.getWriter().println("<p>Nombre: " + nombre + "</p>");
        response.getWriter().println("<p>Edad: " + edad + "</p>");
        response.getWriter().println("</body></html>");
    }
}
```

El método **doGet()** envía los parámetros de forma **explícita** junto a la página, mostrando en la barra de navegación los parámetros y sus valores. Son esas largas cadenas que aparecen en algunas páginas en nuestra barra de navegación, luego del llamado al servlet, por ejemplo:

/buscar?id=1806&valor=0987&texto=todo&...

Las cadenas de la URL tienen la siguiente forma de referencia:

parametro1=valor1¶metro2=valor2&...¶metroN=valorN.

Es decir es una concatenación de pares parámetro-valor a través del operador '&'.

Controlando parámetros de un POST request

En el caso de recibir los parámetros desde un formulario, por ejemplo, se utiliza prácticamente la misma estructura que el método `doGet()`, pero en este caso se denomina **doPost()**. Este método **solo está accesible desde los formularios**. Se envían los parámetros de forma **implícita** junto a la página, es decir, al pasar los parámetros, nosotros no vemos reflejado en ningún sitio qué parámetros son y cuál es su valor.

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // Obtenemos el valor del parámetro "nombre" de la solicitud POST
    String nombre = request.getParameter("nombre");

    // Obtenemos el valor del parámetro "edad" de la solicitud POST
    String edad = request.getParameter("edad");

    // Procesamos y validamos los parámetros según sea necesario

    // Creamos la respuesta para el cliente
    response.setContentType("text/html;charset=UTF-8");
    response.getWriter().println("<html><body>");
    response.getWriter().println("<h1>¡Hola, " + nombre + "!</h1>");
    response.getWriter().println("<p>Tu edad es: " + edad + " años.</p>");
    response.getWriter().println("</body></html>");
}
```

Paso de parámetros de un Servlet hacia una vista JSP

Para pasar parámetros desde un servlet a una página JSP, puedes utilizar el objeto **request** o el objeto **session**. La elección entre uno u otro dependerá de si deseas que los parámetros sean visibles en la URL o si necesitas mantenerlos disponibles a lo largo de la sesión del usuario.

Pasando parámetros usando el objeto request

Puedes agregar atributos al objeto **request** en el servlet y acceder a estos atributos en la página JSP utilizando la expresión **\${}**. Veamos un ejemplo en código utilizando el servlet (**MainServlet.java**):

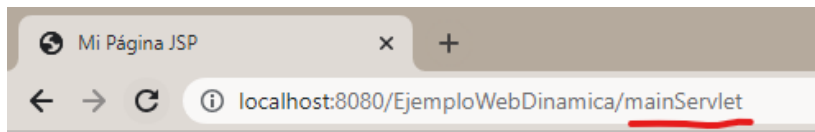
De esta manera en la página JSP (**miPagina.jsp**), puedes acceder al parámetro

```
public class MainServlet extends HttpServlet {  
  
    protected void doGet(HttpServletRequest request, HttpServletResponse response)  
        throws ServletException, IOException {  
        // Obtener el valor del parámetro "nombre" de algún lugar (base de datos, etc.)  
        String nombre = "Juan";  
  
        // Agregar el parámetro "nombre" al objeto request  
        request.setAttribute("nombre", nombre);  
  
        // Redireccionar a la página JSP  
        request.getRequestDispatcher("miPagina.jsp").forward(request, response);  
    }  
}
```

utilizando **\${nombre}**:

```
*miPagina.jsp X  
1  
2 <!DOCTYPE html>  
3 <html>  
4 <head>  
5     <title>Mi Página JSP</title>  
6 </head>  
7 <body>  
8     <h1>Hola, ${nombre}!</h1>  
9 </body>  
10 </html>
```


Con esta configuración, podrás acceder a la vista jsp directamente desde la url del servlet:



Hola, Juan!

Pasando parámetros usando el objeto session

Si necesitas mantener los parámetros disponibles durante toda la sesión del usuario, puedes utilizar el objeto **session** para ello. Los atributos del objeto **session** estarán disponibles hasta que la sesión del usuario expire o se invalide.

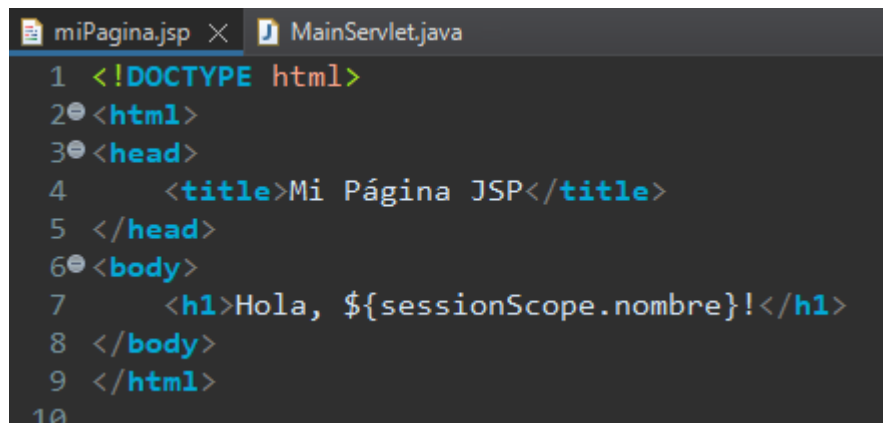
```
protected void doGet(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {
    // Obtener el valor del parámetro "nombre" de algún lugar (base de datos, etc.)
    String nombre = "Juan";

    // Obtener la sesión actual o crear una nueva
    HttpSession session = request.getSession(true);

    // Agregar el parámetro "nombre" al objeto session
    session.setAttribute("nombre", nombre);

    // Redireccionar a la página JSP
    response.sendRedirect("miPagina.jsp");
}
```

De esta manera en la página JSP (**miPagina.jsp**), puedes acceder al parámetro utilizando **`${sessionScope.nombre}`**:



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Mi Página JSP</title>
5 </head>
6 <body>
7   <h1>Hola, ${sessionScope.nombre}!</h1>
8 </body>
9 </html>
10
```

En ambos casos, los parámetros pasados desde el servlet estarán disponibles en la página JSP y podrás mostrarlos o utilizarlos según tus necesidades. Es importante recordar que los atributos agregados al objeto **request** solo estarán disponibles durante la ejecución de la solicitud actual, mientras que los atributos agregados al objeto **session** estarán disponibles durante toda la sesión del usuario.

Implementando un formulario con validación en el servlet

Como hemos visto, el método que utilizamos para recuperar los datos de los parámetros es **getParameter("X")**. Esta recuperación puede generar resultados diferentes:

- Cuando el campo "X" no existe, obtenemos el valor **null**.
- Cuando el campo "X" existe, pero no tiene valor, obtenemos el valor vacío (**isEmpty**).
- Cuando el campo "X" existe y tiene valor, obtenemos la cadena de caracteres que corresponde a dicho valor.

La respuesta ante cada una de las situaciones depende de la lógica deseada en la aplicación. Es habitual requerir que determinados campos tengan valor (campos requeridos), y en muchos casos es deseable que los valores de los campos cumplan determinadas propiedades (ej.- que el RUT introducido sea un valor numérico con el número de dígitos correcto).

```
protected void doPost(HttpServletRequest request, HttpServletResponse response)
    throws ServletException, IOException {

    // Obtener los parámetros enviados desde el formulario
    String nombre = request.getParameter("nombre");
    String email = request.getParameter("email");

    // Realizar la validación

    if (nombre == null || nombre.trim().isEmpty() || email == null || email.trim().isEmpty()) {
        // Si hay campos vacíos, redirigir al usuario de vuelta al formulario con un
        // mensaje de error
        response.sendRedirect("registro.html?error=1");
    } else {
        // Si la validación es exitosa, redirigir al usuario a una página de éxito
        response.sendRedirect("exito.html");
    }
}
```

Para asegurarnos de que los datos que se ingresaron en el formulario son correctos o aceptables, vamos a realizar la validación en el servlet. En la vista JSP tenemos que tener en cuenta de marcar con el atributo **required** los inputs cuyos datos sean requeridos para procesar.

```
<!DOCTYPE html>
<html>
<head>
    <title>Formulario de Registro</title>
</head>
<body>
    <h1>Registro de Usuario</h1>
    <form action="registroServlet" method="post">
        <label for="nombre">Nombre:</label>
        <input type="text" name="nombre" id="nombre" required>
        <br>
        <label for="email">Email:</label>
        <input type="email" name="email" id="email" required>
        <br>
        <input type="submit" value="Registrarse">
    </form>
</body>
</html>
```

Cierre

En esta lección, hemos recorrido los conceptos fundamentales sobre Servlets en el desarrollo web con Java. Hemos explorado cómo los Servlets y los Contenedores Web nos brindan la capacidad de crear aplicaciones dinámicas y eficientes.

Hemos aprendido a trabajar con Sesiones y Cookies, permitiéndonos mantener información entre solicitudes y mejorar la experiencia del usuario. Asimismo, hemos descubierto cómo compartir datos entre Servlets, lo que nos ha permitido crear aplicaciones más robustas y escalables.

También vimos cómo enviar datos desde un Servlet hacia una vista JSP y cómo implementar formularios con validación en el Servlet. Estas habilidades son esenciales para construir aplicaciones web dinámicas y efectivas con Java.

¡Nos vemos de nuevo, más adelante! 

Referencias

- RicardoGeek. (s.f.). Manejo de sesiones y cookies en Servlets Java. Recuperado el 28 de julio de 2025, de <https://ricardogeek.com/manejo-de-sesiones-y-cookies-en-servlets-java/>
- Universitat Politècnica de València. (s.f.). Apuntes de la sesión 9: JSP [Curso de J2EE 2006–2007]. Recuperado el 28 de julio de 2025, de <http://www.jtech.ua.es/j2ee/2006-2007/restringido/jsp/sesion09-apuntes.html>
- Cuervo, V. (2007, 25 de diciembre). Recibir parámetros en un Servlet. Línea de Código. Recuperado el 28 de julio de 2025, de <https://lineadecodigo.com/java/recibir-parametros-en-un-servlet/>
- Chuidiang. (s.f.). Pasar datos entre JSPs y Servlets: Page, Request, Session y Application scope. Recuperado el 28 de julio de 2025, de https://chuidiang.org/index.php?title=Pasar_datos_entre_JSPs_y_Servlets._Page._Request._Session_y_Application_scope
- Seabrokewindows. (año). Procesos concurrentes de un objeto Servlet. Recuperado el día mes año, de <https://ejemplo.com/tu-recurso>

¡Muchas gracias!

Nos vemos en la próxima lección

